



CSE422

Course Title: Artificial Intelligence

Semester : Spring_2025

Project Title: Predicting Income Level with Machine Learning

Member Details:

Shahriar Mohammad	ID: 22299020	Section: 15
Nur Faisal Ahmed Sourove	ID: 24141094	Section: 13

Project Demonstration Date: 14 May 2025

Instructors: Pollock Nag [PLN] & Aunon Halder [CANH]

Table of Content

Introduction	Page-1
Dataset Description	Page-1
Dataset pre-processing	Page-4
Dataset Splitting	Page-4
Model training & testing	Page-4
Model selection/ Comparison Analysis	Page-5
Conclusion	Page-7

- **Introduction**

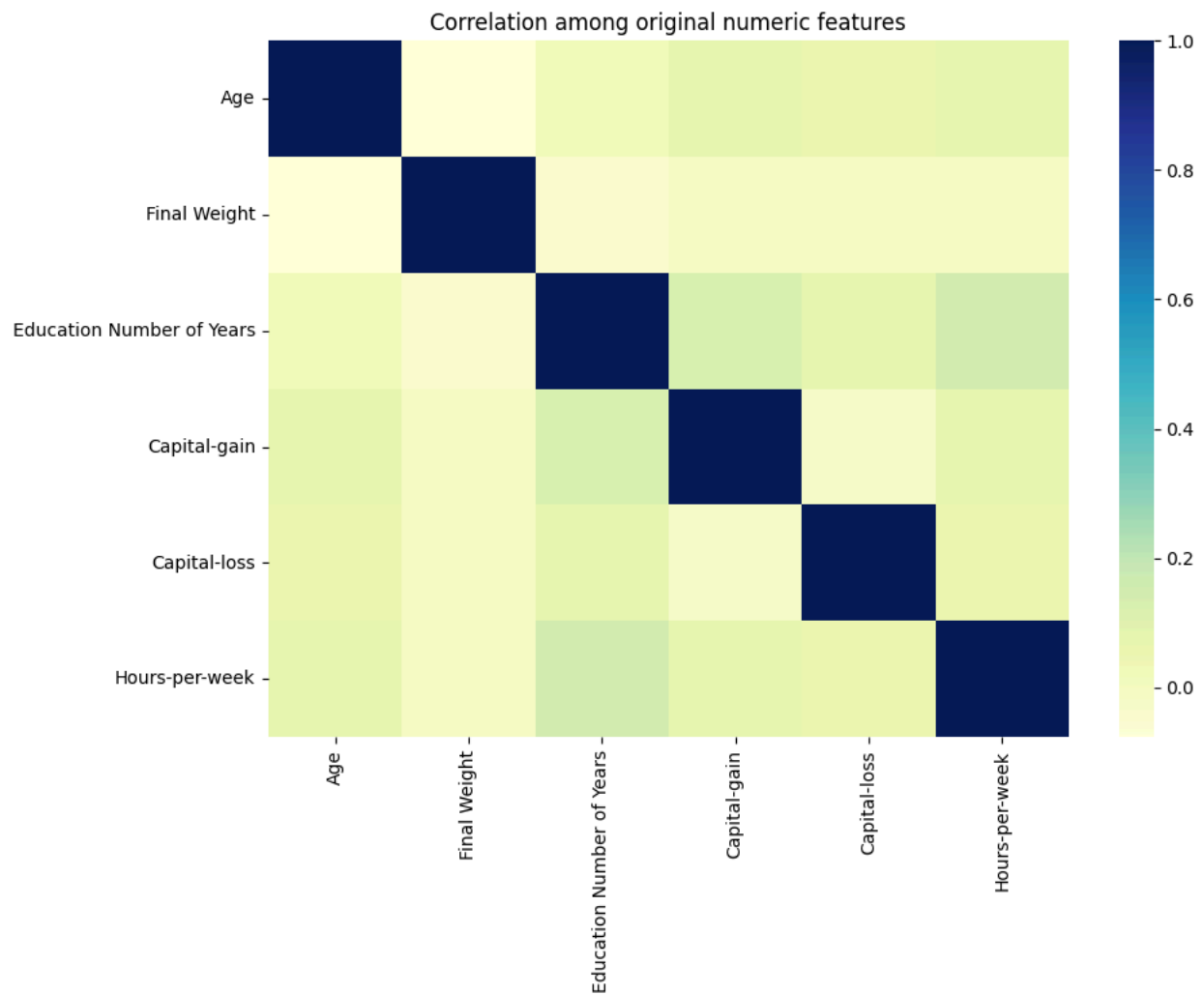
The main goal of the project is to use machine learning to predict whether a person earns more or less than \$50k per year, based on different types of data like education, job type and hours worked. The Adult Income dataset that we are using today was collected from the U.S. Census. The main target of our project is to build an accurate mode and to understand which factors affect income the most. Using the resources of machine learning provided by our instructors during the lab classes we will be cleaning the data, exploring it and testing three models which are Neural Network, Logistic Regression and K-Nearest Neighbours. We will be comparing their outputs using precision, accuracy, recall, F1-score and AUC.

- **Dataset description**

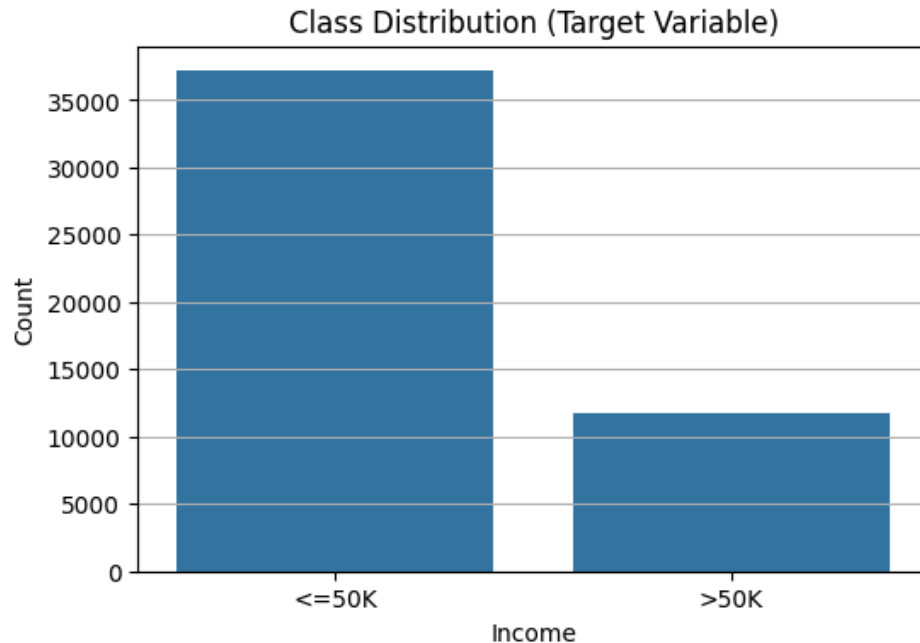
- Dataset Description

- The dataset has 14 input features and 1 target column, making a total of 15 features.
 - This is a classification problem because the target variable has only two possible outcomes which are values that are either greater than or equal to 50K or less than 50K.
 - The dataset contains approximately 32,561 rows.
 - We have 2 types of features:
 1. Quantitative features:
 - Age
 - Final Weight
 - Education Number of Years
 - Capital-gain
 - Capital-loss
 - Hours-per-week
 2. Categorical Features:
 - Workclass
 - Education
 - Marital-status
 - Occupation
 - Relationship
 - Race
 - Sex
 - Native-country
 - Most features have low correlation with each other. The strongest correlation is seen in Education Number of Years, Capital-gain and Hours-per-week.

These typically show positive correlation with earning >50K. That means people with more education, who receive capital gains and those who work more hours per week are more likely to fall into the >50K income bracket.



- No pairs of features show strong correlation, so no features need to be dropped due to multicollinearity.
- Imbalanced Dataset
 - The output feature has two classes:
 - $\leq 50K$ (lower income)
 - $> 50K$ (higher income)



- As show in the class distribution bar chart, the dataset is imbalance
 - Around 75% of the data falls into the <=50K class
 - Only about 25% is in the >50K class
 - This imbalance can affect model performance for recall and precision on the minority class.
 - To improve this we used stratified train-test split to prevent the class ratio and class-weight balancing to avoid bias towards the majority class.
- Perform exploratory data analysis to extract some important relationships from your data. We performed several EDA steps to better understand the structure and relationships within the dataset:
 - Missing Value Handling:
The dataset originally contained missing values represented as "?". These were found in columns like Workclass, Occupation, and Native-country. We replaced them with NaN and imputed categorical features using the most frequent value and numerical features using the median..
 - Class Distribution:
The target variable is imbalanced, with about 76% of the samples labeled <=50K and 24% labeled >50K. This was visualized using a bar chart. The imbalance was addressed through stratified splitting and class-weighted models.
 - Correlation Analysis:
 - A heatmap of the numerical features showed:
 - No multicollinearity (no correlation > 0.85)
 - Weak but meaningful correlations between:

The dataset originally contained missing values represented as "?". These were found in columns like Workclass, Occupation, and Native-country. We replaced them with NaN.

- **Dataset pre-processing**

- Several columns in the dataset contained placeholder values like "?" instead of a proper missing value indicator. These placeholders were not automatically recognized as missing by pandas and would break encoding. We replaced "?" with actual NaN values using `.replace('?',np.nan)` and applied most frequent imputation for categorical features and median imputation for numerical features. This ensured that no rows were deleted, and outlier influence on numerical imputation was minimized.
- Most of the features were categorical, such as Workclass, Education, Occupation, etc., and could not be used directly in machine learning models. Machine learning algorithms require numeric input. We applied two types of encoding which are Label Encoding for binary columns and One-Hot Encoding for multi-class categorical columns.
- Many numeric features had different ranges and contained outliers. Unscaled features negatively affect distance-based models like KNN and also slow down gradient-based models like Neural Networks. We used RobustScaler, which scales features using the interquartile range (IQR) and is robust to outliers. Only the numeric columns were scaled, the encoded 0/1 columns were left untouched.

- **Dataset splitting**

- To evaluate model performance properly, we split the dataset into training and test sets using a stratified split. Since the target variable is imbalanced, stratification ensures that both subsets maintain the original class distribution.
- Train set: 70%
- Test set: 30%
- Split type: Stratified random split

- **Model training & testing**

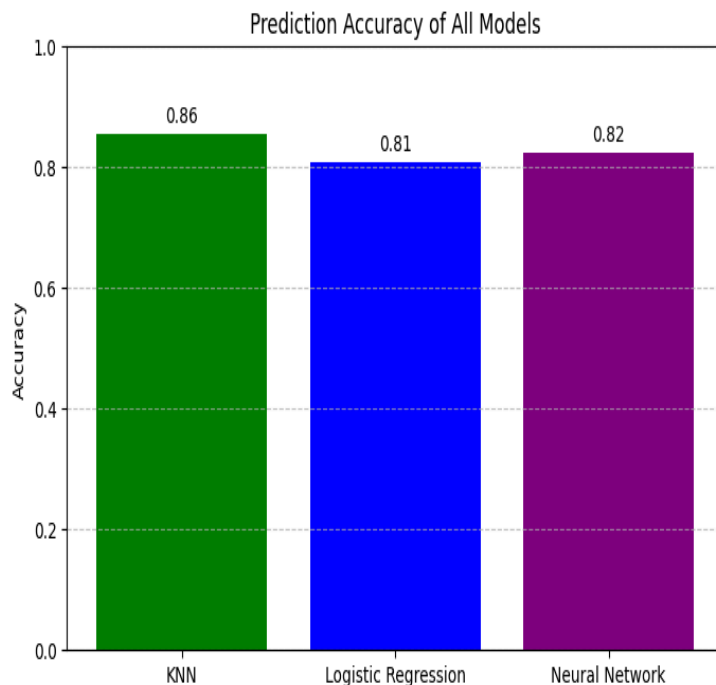
We applied 3 classification models which are Neural Network, Logistic Regression and K-Nearest Neighbour.

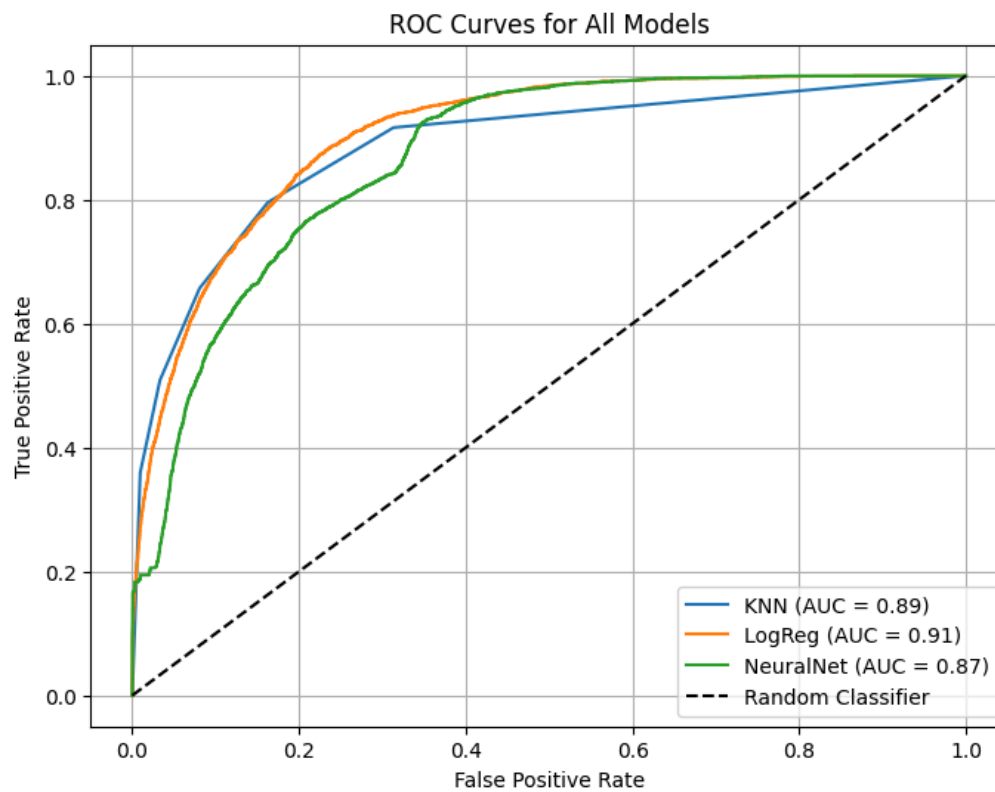
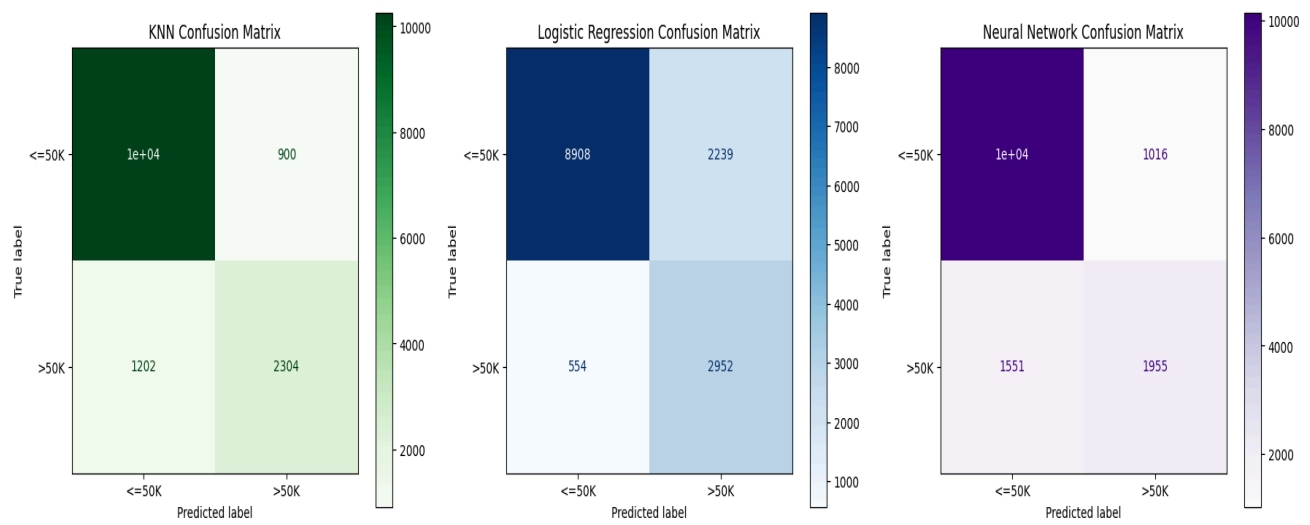
- KNN is a simple, non-parametric, instance-based learning method. It classifies a data point based on the majority class among its "k" closest neighbours. It's a good baseline for classification tasks. We used KNeighborsClassifier with scaled features and evaluated it on accuracy, precision, recall, F1 and AUC. Scaling was necessary as KNN relies on distance between points.
- Logistic Regression is a widely used linear classifier. It offers both strong performance and interpretability, allowing us to understand the influence of each feature on the prediction. We trained a LogisticRegression model with class weights balanced to handle the income imbalance. We used Robust Scaling on numeric features to reduce outlier influence. Performance was evaluated using the same metrics.

- Neural Networks can capture complex, non-linear relationships in the data. It allows us to compare modern deep learning methods with traditional approaches. We implemented a feed-forward neural network using TensorFlow/Keras with 2 hidden layers, ReLU activation, dropout regularization and sigmoid output for binary classification.
- We trained on the scaled dataset and used early stopping to avoid overfitting. Evaluation was done using accuracy, F1score and ROC-AUC.

- **Model selection/Comparison analysis**

KNN Classification Report:				
	precision	recall	f1-score	support
<=50K	0.90	0.92	0.91	11147
>50K	0.72	0.66	0.69	3506
accuracy			0.86	14653
macro avg	0.81	0.79	0.80	14653
weighted avg	0.85	0.86	0.85	14653
Logistic Regression Classification Report:				
	precision	recall	f1-score	support
<=50K	0.94	0.80	0.86	11147
>50K	0.57	0.84	0.68	3506
accuracy			0.81	14653
macro avg	0.76	0.82	0.77	14653
weighted avg	0.85	0.81	0.82	14653
Neural Network Classification Report:				
	precision	recall	f1-score	support
<=50K	0.87	0.91	0.89	11147
>50K	0.66	0.56	0.60	3506
accuracy			0.82	14653
macro avg	0.76	0.73	0.75	14653
weighted avg	0.82	0.82	0.82	14653





To compare the performance of the three classification models which are KNN, Logistic Regression, and Neural Network, we evaluated them using multiple metrics:

- Accuracy:
 - A bar chart was used to visualize prediction accuracy:
 - KNN performed best in terms of accuracy (0.86)
 - Neural Network and Logistic Regression followed with 0.82 and 0.81 respectively
- Precision, Recall, and F1-score:
 - KNN had the highest F1-score for both classes combined (especially strong on the majority class)
 - Logistic Regression achieved better recall on the >50K class
 - Neural Network maintained balance between precision and recall but had slightly lower recall on the minority class
- Confusion Matrices
 - We plotted confusion matrices for all models:
 - KNN produced fewer false positives than the other models
 - Logistic Regression captured more true positives for >50K, but misclassified more <=50K
 - Neural Network was more balanced but slightly less effective on the minority class
- ROC Curve and AUC Score
 - All ROC curves were plotted together. Based on AUC:

Logistic Regression scored highest (0.91)

KNN followed (0.89)

Neural Network scored (0.87)

- **Conclusion**

From the results, we can understand that different models performed better on different aspects of the classification task. K-Nearest Neighbours (KNN) gave the best overall accuracy, while Logistic Regression achieved the highest AUC score, meaning it was best at ranking predictions. The Neural Network was reasonably balanced but slightly underperformed in recall for the high-income class. From the results, we can understand that different models performed better on different aspects of the classification task. K-Nearest Neighbours (KNN) gave the best overall accuracy, while Logistic Regression achieved the highest AUC score, meaning it was best at ranking predictions. The Neural Network was reasonably balanced but slightly underperformed in recall for the high-income class. The challenges that we faced were handling missing values which were not standard, class imbalance in the target, which affected recall for the minority class, scaling only numeric columns while preserving one-hot encoded categories and training time and threshold tuning for the neural network. Despite these challenges, the overall classification performance was strong across all models, and the results reinforce

the importance of proper preprocessing and algorithm selection in solving real-world machine learning problems.